

Lab-01: ARM Assembly and Keil uVision5

1. Objectives

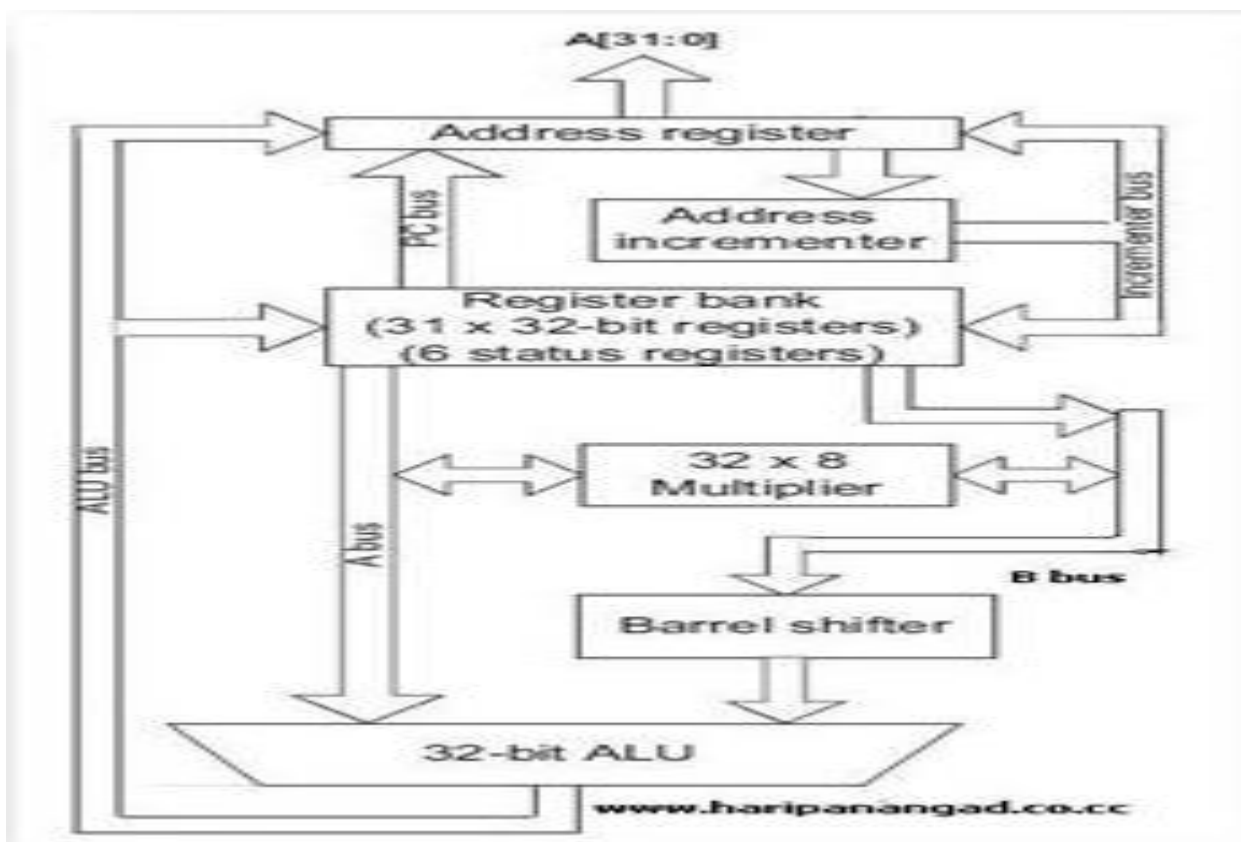
- Introduce some of the ARM architecture to students.
- Begin to use the lab tool - Keil uVision 5.
- The students will create a project and write an ARM assembly language program based on a simulated target.

2. Introduction

- The ARM (Advanced RISC Machine) architecture is introduced in the class (also see <http://www.arm.com>.) Keil MDK-ARM is a complete software development toolkit for ARM processor-based microcontrollers. Keil uVision5 will be used in the lab. The ARM Cortex-M3 processor will be examined with the STM32VLDISCOVERY board. The following is some important information for you.

3. ARM Architecture

ARM processors are mainly used for low-power and low cost applications such as mobile phones, communication modems, automotive engine management systems, and hand-held digital systems. Here is a diagram of the ARM architecture for your reference.



ARM Architecture is an Enhanced RISC Architecture. It has large uniform Register file and uses Load Store Architecture. i.e. operations operate on registers and not in memory locations.

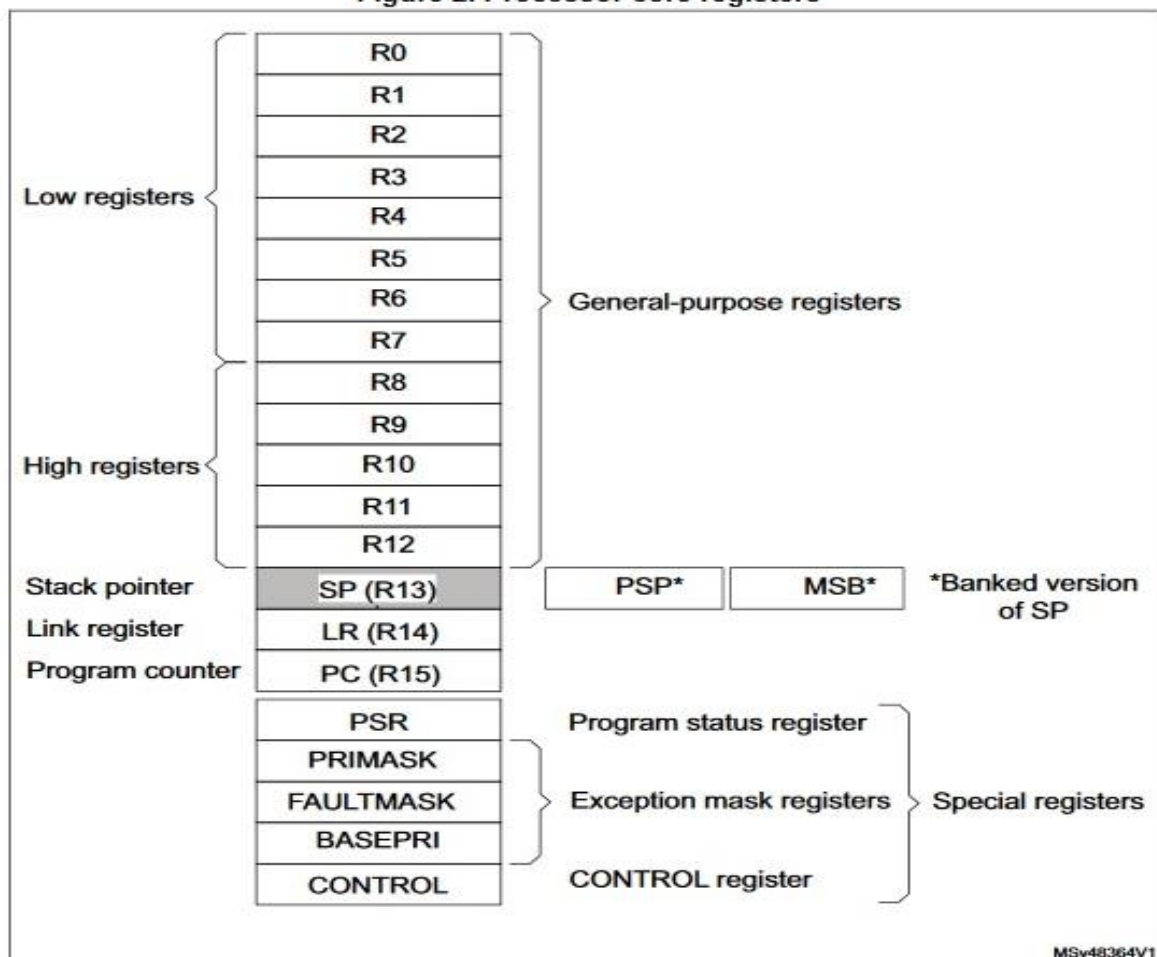
ARM Architecture instructions are of uniform and fixed length. It is a 32 bit processor. It also has 16 bit variant called THUMB. i.e. it can be used as 32 bit and as 16 bit processor.

- ARM cores are licensed to partners/manufacturers so as to develop and fabricate new microcontrollers around same processor cores. A microcontroller is a small computer on a single integrated circuit containing a processor core, memory, and programmable input/output peripherals. The ARM Cortex-M3 microcontroller will be used in the lab with the STM32VLDISCOVERY board.

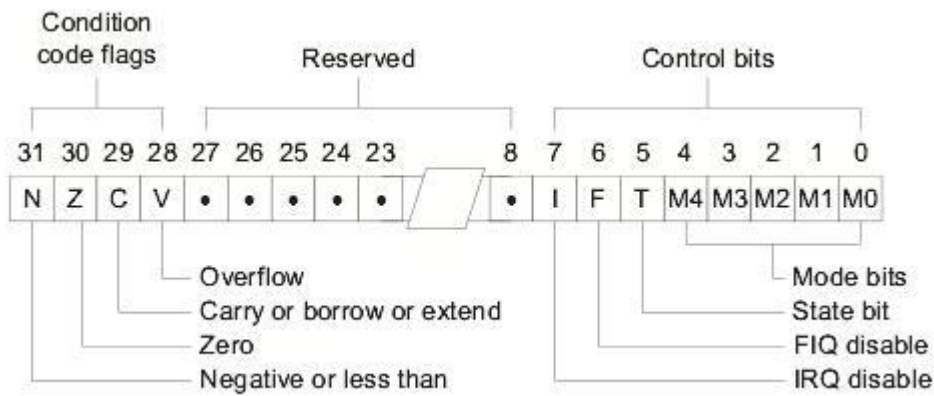
4. ARM Cortex-M3 Core Registers

Core registers

Figure 2. Processor core registers

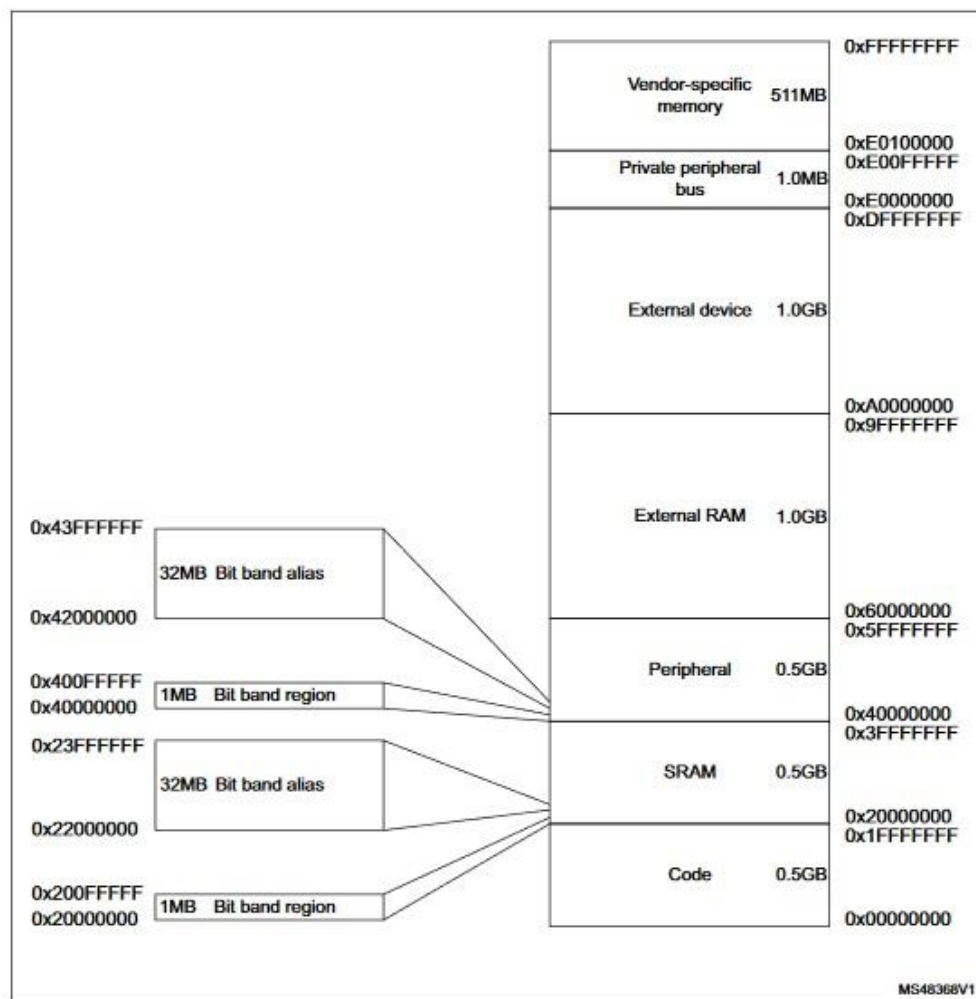


- Here is the Program Status Register Format:



- Condition code flags in CPSR:
- N - Negative or less than flag
- Z - Zero flag
- C - Carry or borrow or extended flag
- V - Overflow flag
-

5. ARM Cortex-M3 Memory Map



6. Create an ARM Assembly Language Program

To create an assembly language program, you need to use a text editor such as **NotePad** in Microsoft Windows environment. There is a text edit in the Keil uVision5 for you to use too. The file name must have a **.s** at the end. Let's look at the following program called **FirstArm.s** on a PC. The file **FirstArm.s** contains the source code of the program to load registers and demonstrate a few other operations. We will use Keil uVision5 to create a project and execute this program so that you can get a feel of how Keil uVision5 works.

```

;The semicolon is used to lead an inline documentation.
;This is the first ARM Assembly language program you see in
the lab.
;This program skeleton was from Dave Duguid and Trevor
Douglas.
;When you write your program, you could have your info at the
top document
;;; Directives
        PRESERVE8
        THUMB

; Vector Table Mapped to Address 0 at Reset
; Linker requires __Vectors to be exported

        AREA      RESET, DATA, READONLY
        EXPORT    __Vectors

__Vectors
        DCD      0x20001000
                ; stack pointer value when stack is empty
                ;The processor uses a full descending stack.
                ;This means the stack pointer holds the address of
the last
                ;stacked item in memory. When the processor pushes
a new item
                ;onto the stack, it decrements the stack pointer
and then
                ;writes the item to the new memory location.

        DCD      Reset_Handler ; reset vector

        ALIGN

; The program
; Linker requires Reset_Handler

        AREA      MYCODE, CODE, READONLY

        ENTRY

```

```

EXPORT Reset_Handler

Reset_Handler
;;;;;;;;;;User Code Starts from the next line;;;;;;;;;;

    MOV R0, #12

STOP

    ADD R0, R0, #4
    B STOP

    END      ;End of the program

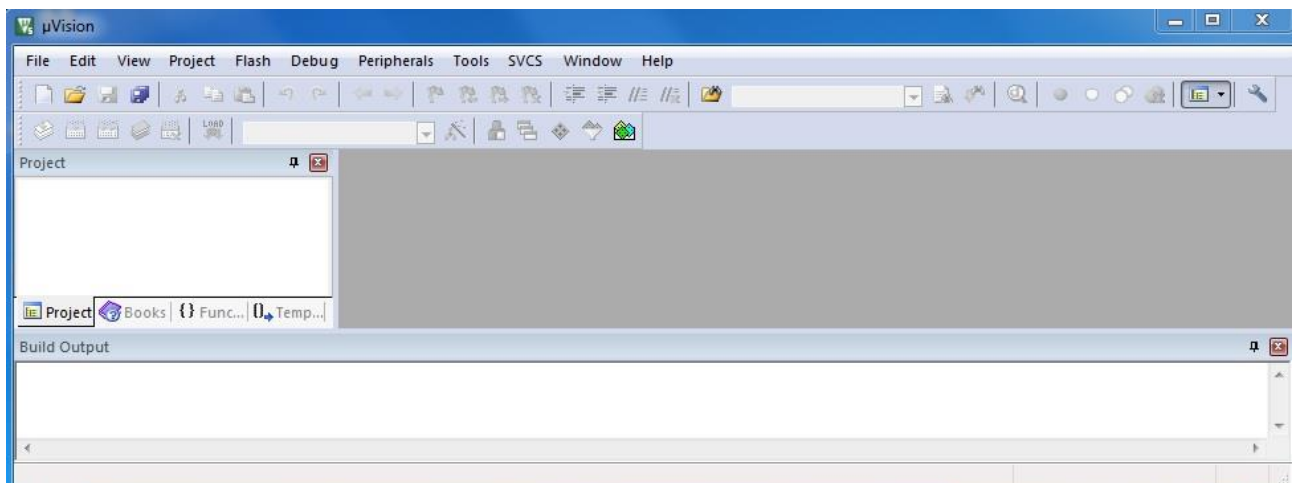
```

7. Start up Keil uVision5

- Before you start up, you are recommended that you create a folder to hold all your project files.
- For example: you can have a folder "FirstARM-Project" ready before hand.

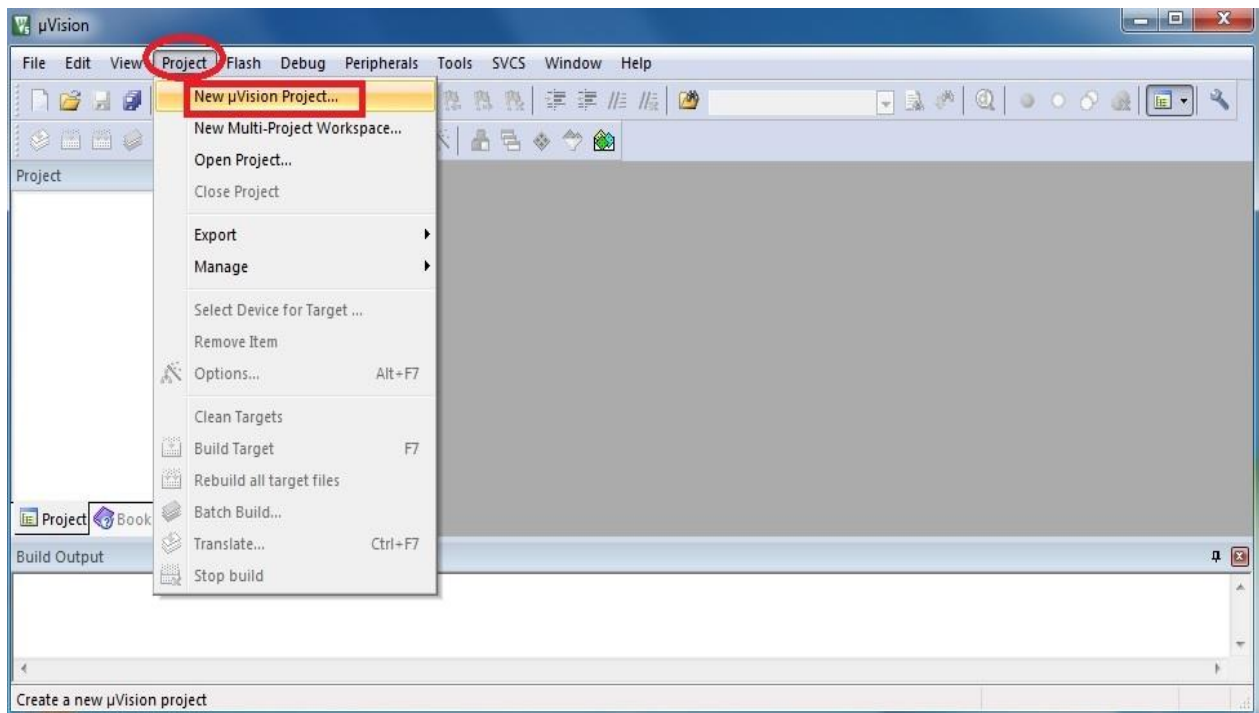


You can start up **uVision5** by clicking on the icon from the desktop or from the "Start" menu or "All Programs" on a lab PC. The following screen is what you will see.

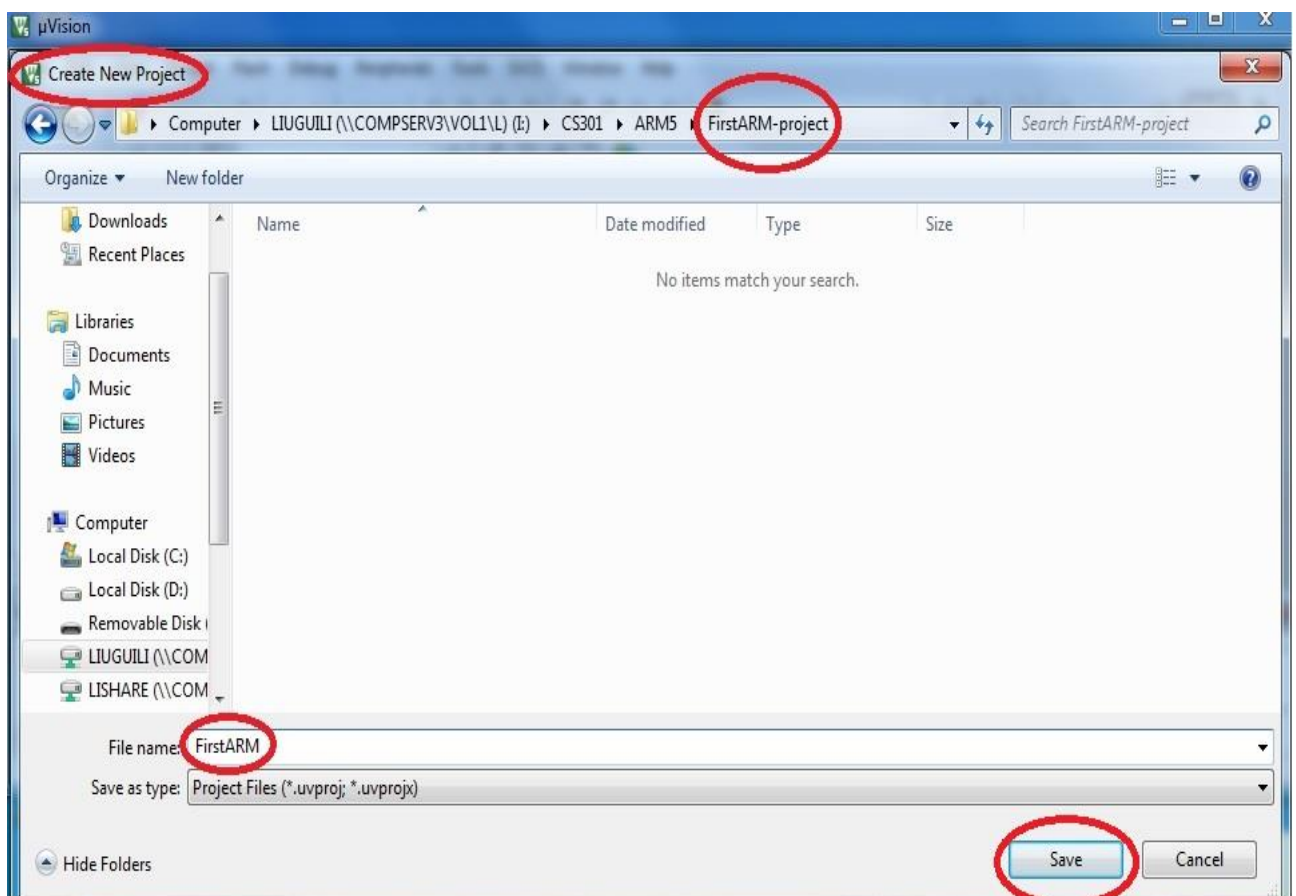


7.1. Create a project

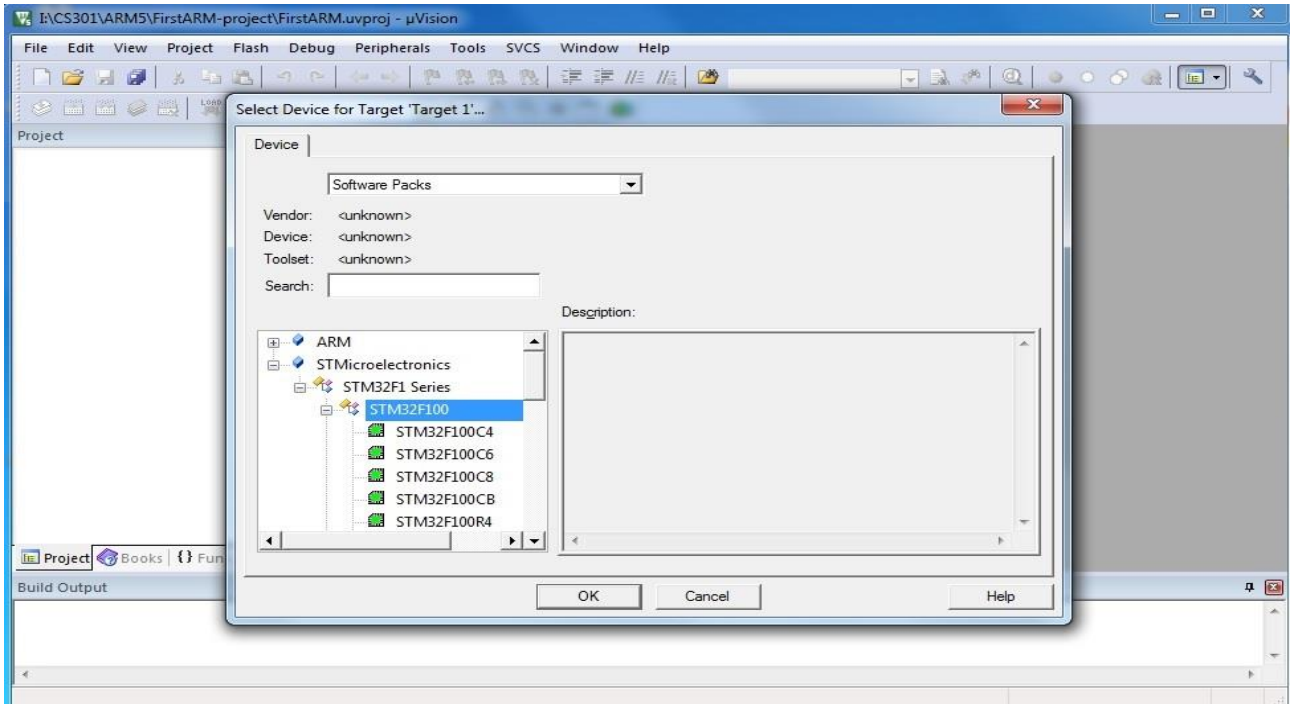
Let's create our first ARM **uVision5** project now. To create a project, click on the "Project" menu from the **uVision5** screen and select "New uVision Project...".



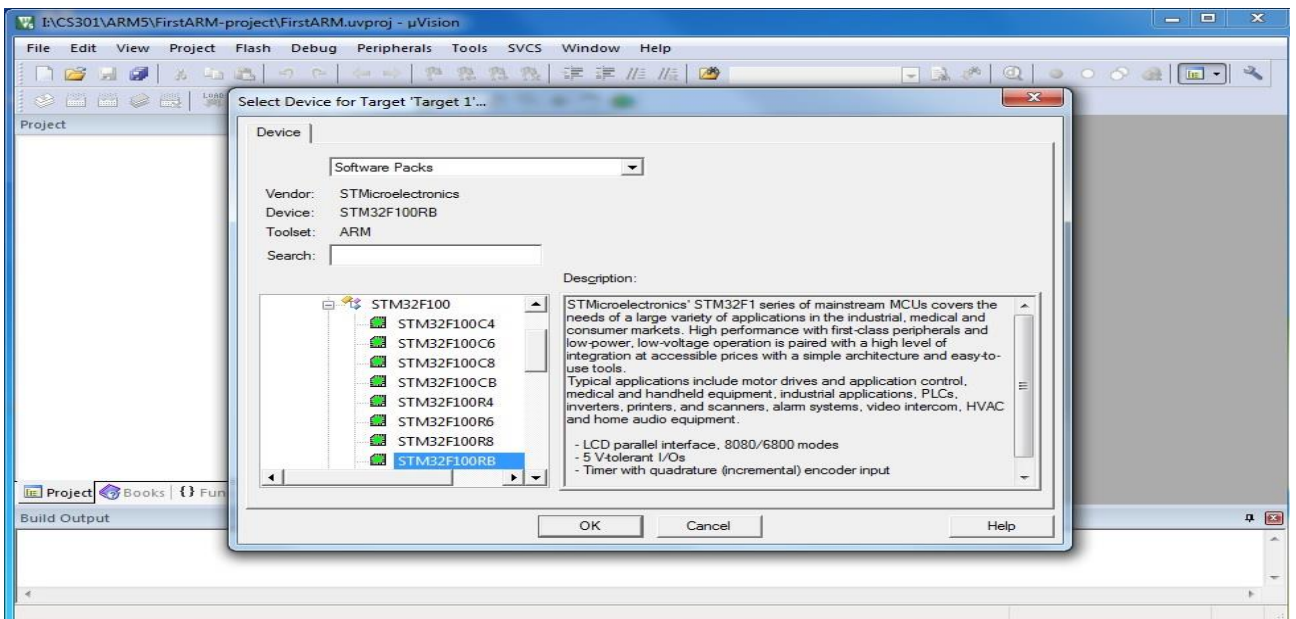
Then, select the folder that you prepared for, give project a name and save it.



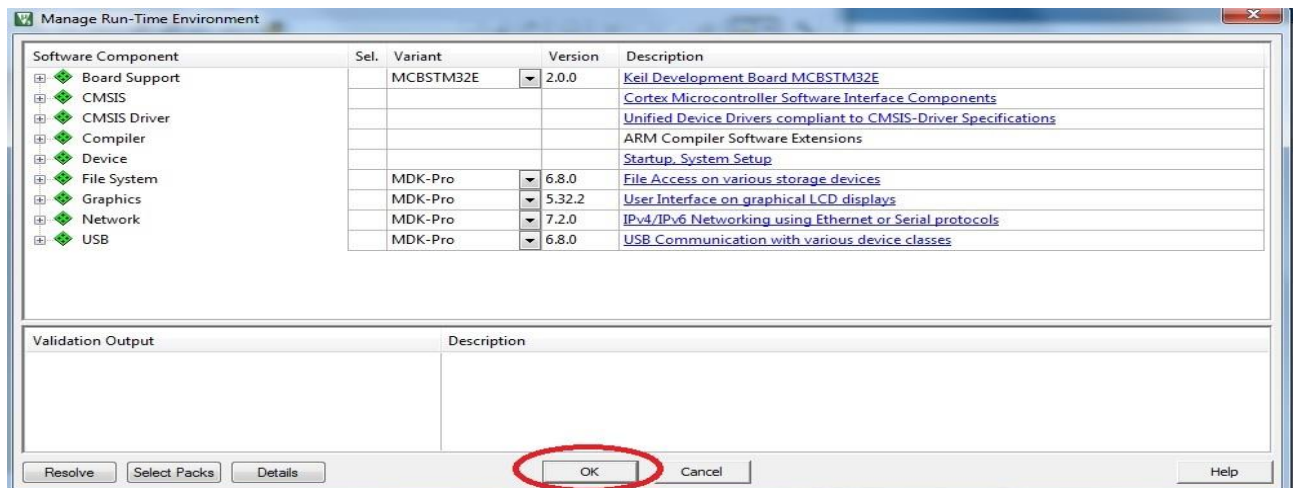
From the "Select Device for Target" window, select "STMicroelectronics" and then "STM32F1Series".



click on "+" beside "STM32F100" and then select "STM32F100RB" and click on "OK".

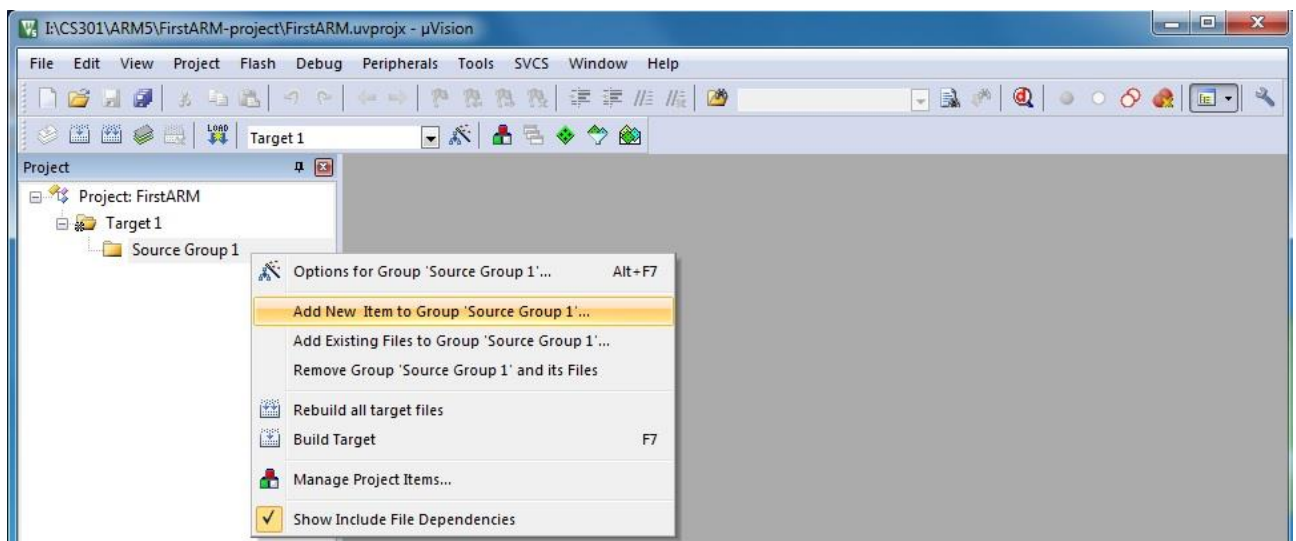


Make sure you click on "OK" for the following pop up window.

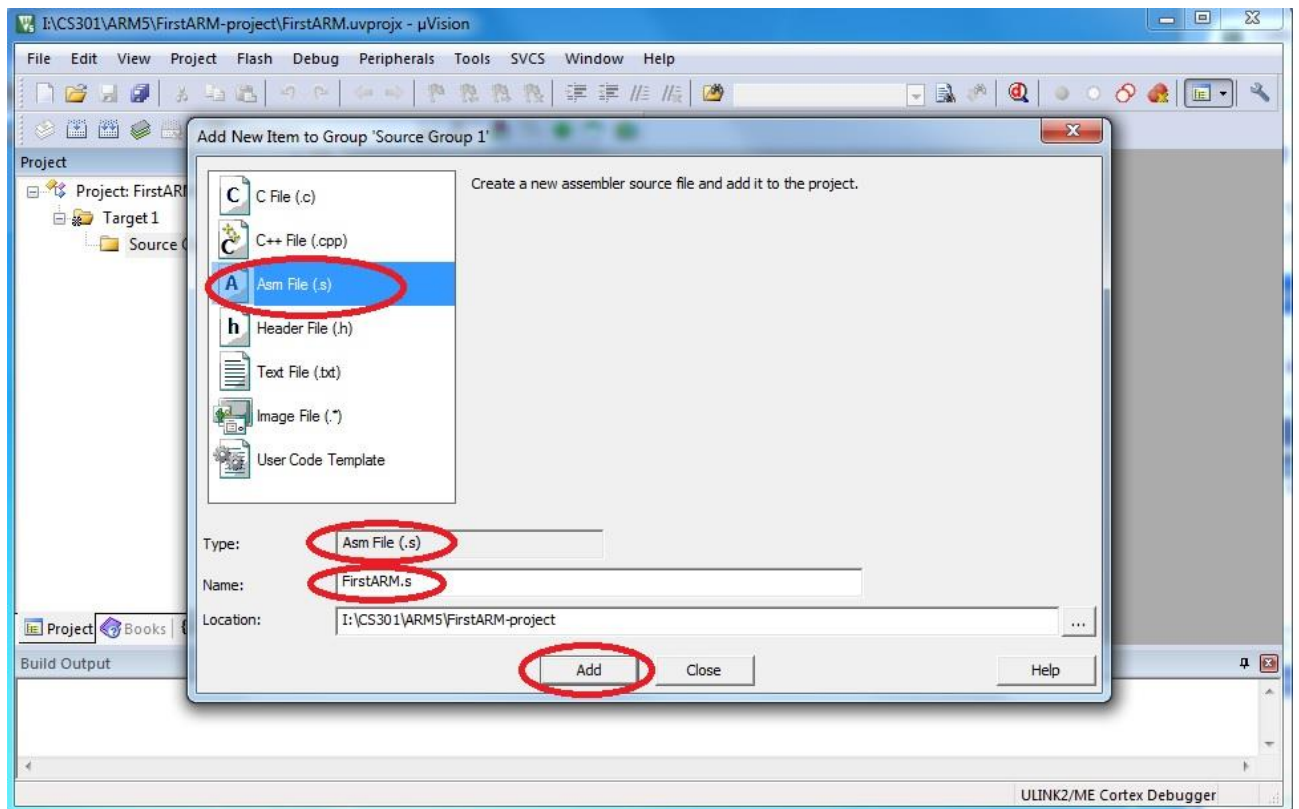


7.2. Create Source File and Add Source File to the Project

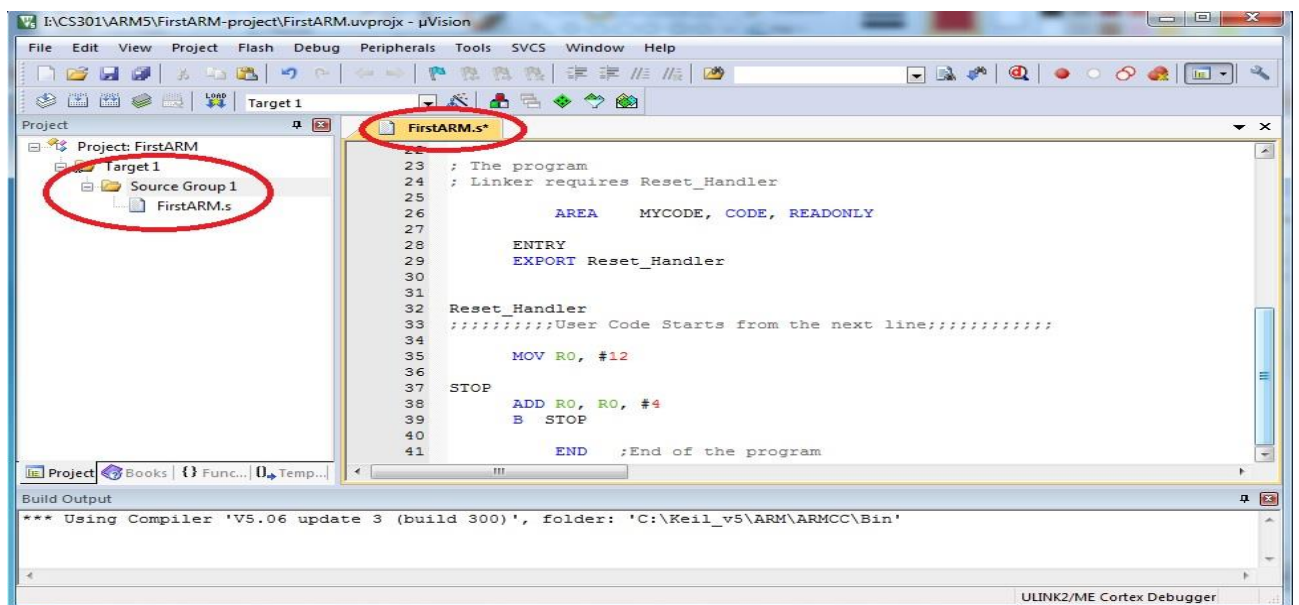
Right click on "Source Group 1" and then select "Add New Item to Group 'Source Group 1'..."



You will see the following window and make the suggested selections to proceed.



You will see the "FirstARM.s*" text edit window. That is the place you will write your ARM Assembly language program. For a test, you can copy and paste the example program into this window. You can click on the "save" button to save your project.



– Write the following ASM program

```
PRESERVE8
THUMB
; Vector Table Mapped to Address 0 at Reset
; Linker requires __Vectors to be exported
```

```

AREA      RESET, DATA, READONLY
EXPORT    __Vectors

```

```

__Vectors
    DCD    0x20001000
    DCD    Reset_Handler ; reset vector
    ALIGN

```

```

AREA      MYCODE, CODE, READONLY

```

```

ENTRY
EXPORT    Reset_Handler

```

```

Reset_Handler
;;;;;;;;;;User Code Starts from the next line;;;;;;;;;;

```

```

    MOV    R0, #12

```

```

STOP

```

```

    ADD    R0, R0, #4
    B      STOP

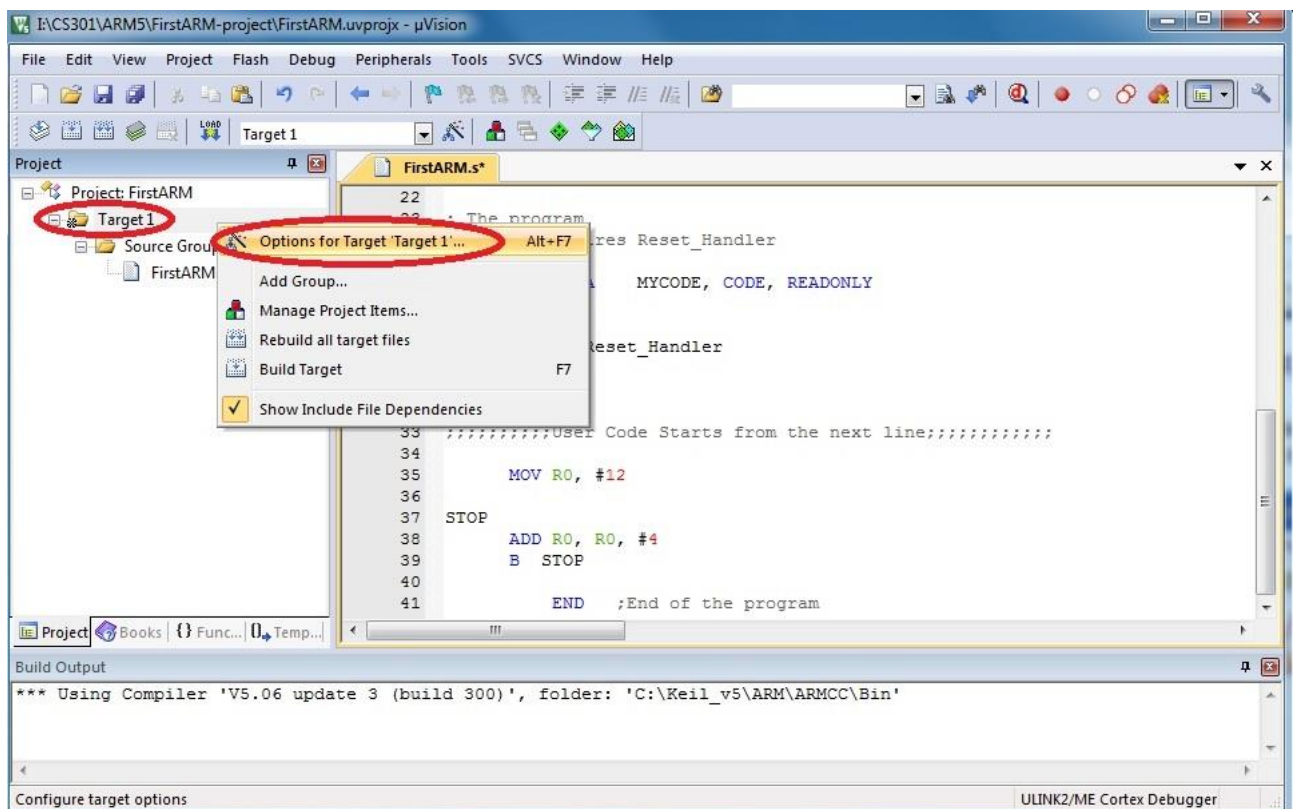
```

```

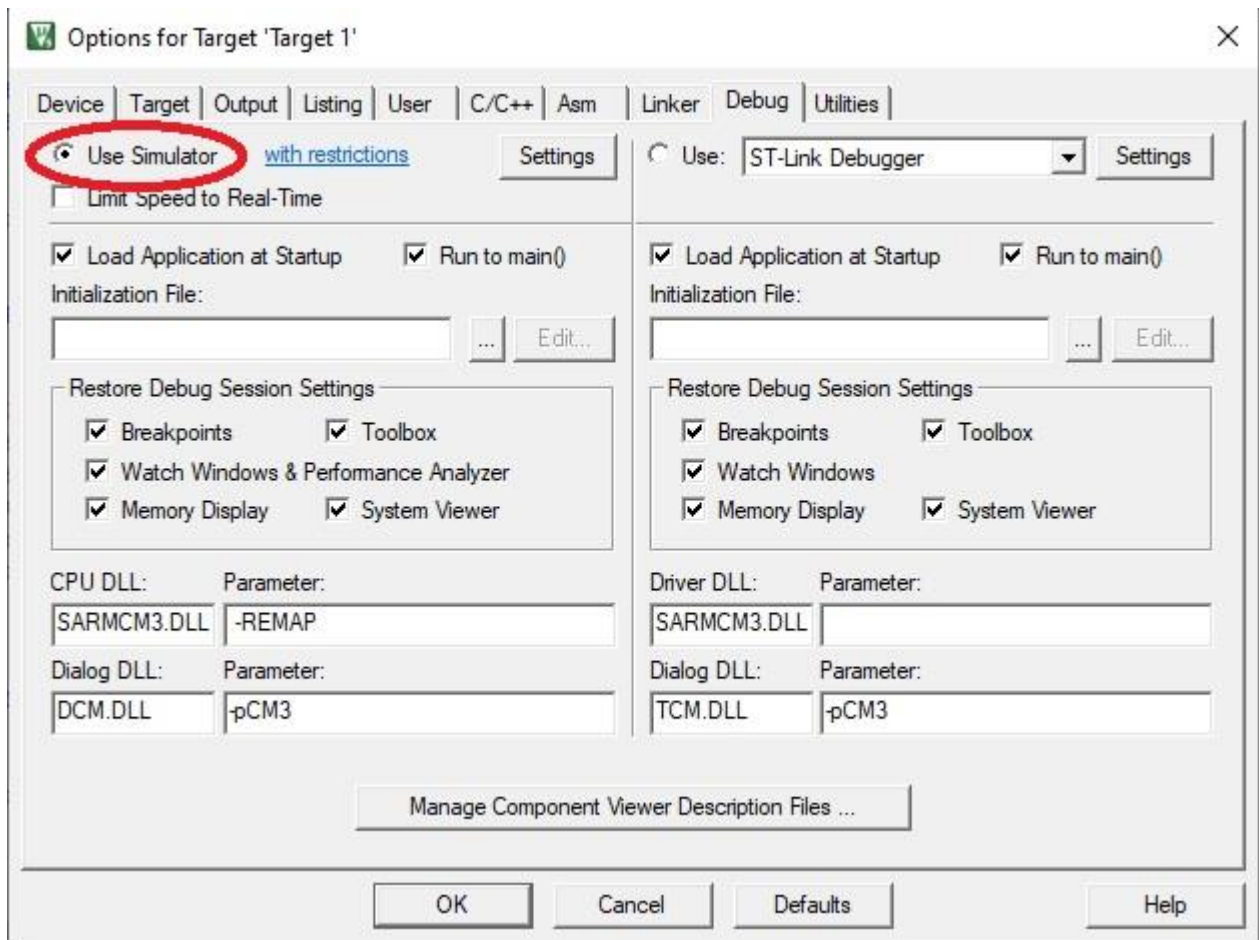
END      ;End of the program

```

You can right click on "Target 1" and then select "options for Target 'Target 1'..." the same as the following screen.

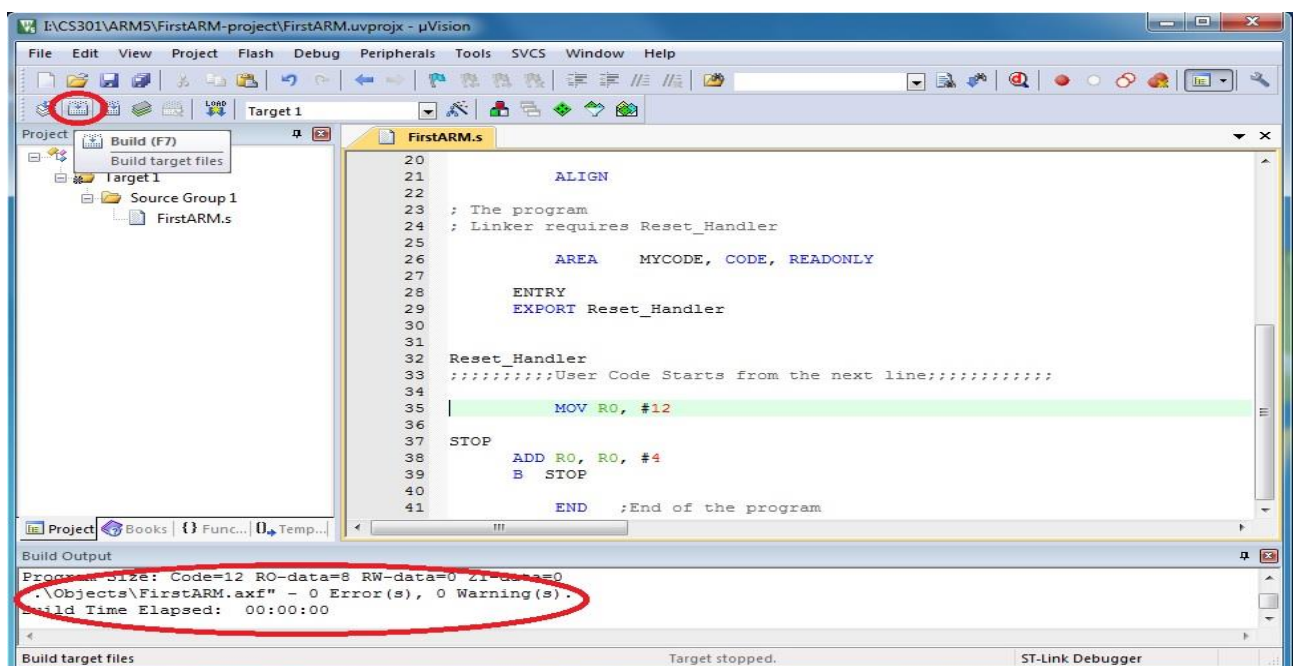


Please click on "Debug" and then select "Use Simulator".



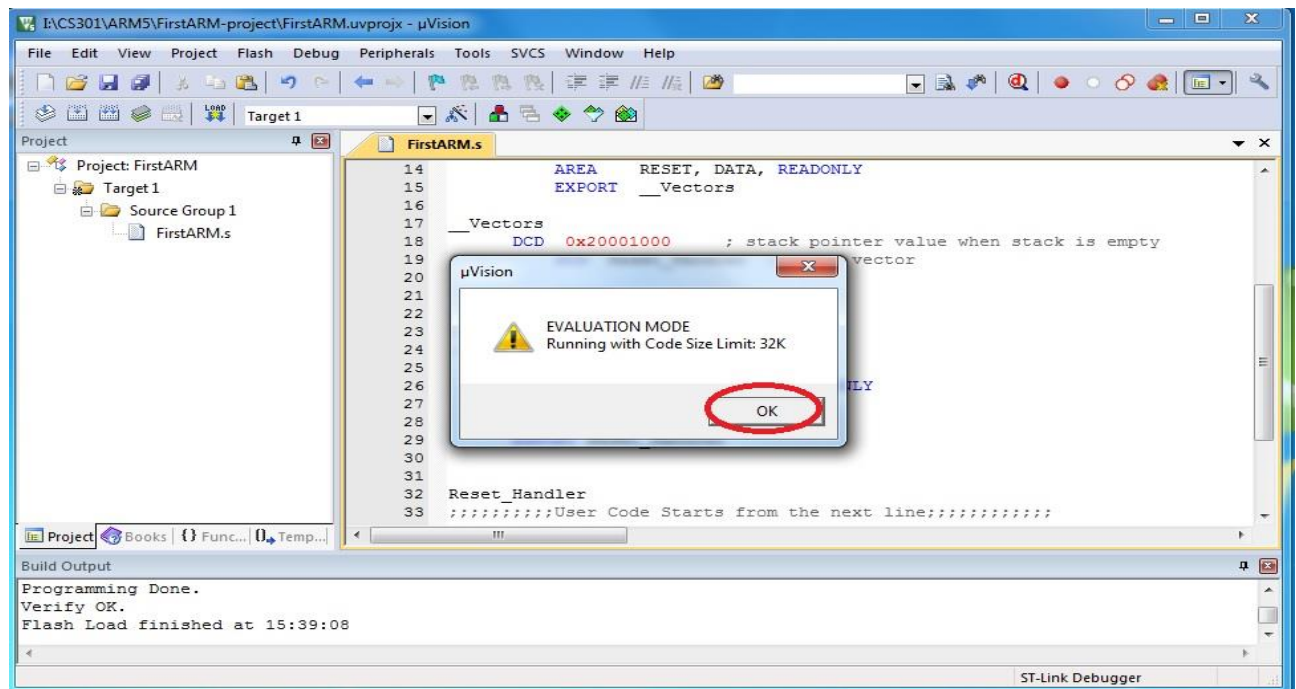
7.3. Build your project

Click on the "Build" button or from the "Project" menu, you will see the following screen.



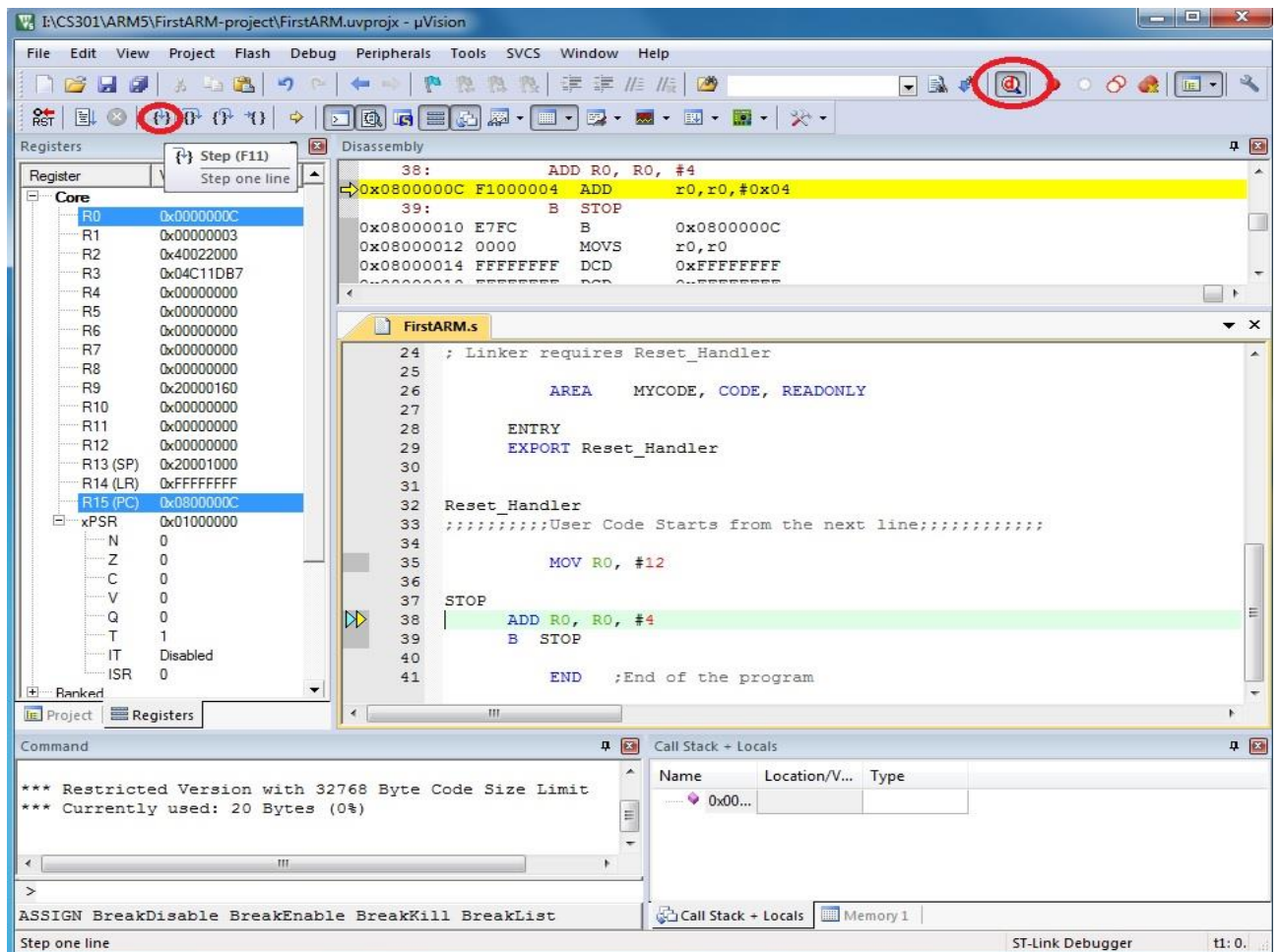
7.4. Run the program in your project

When the assembler is happy with the program, we can run the program by selecting "Start/Stop Debug Session" from the "Debug" menu or clicking on the debug button.



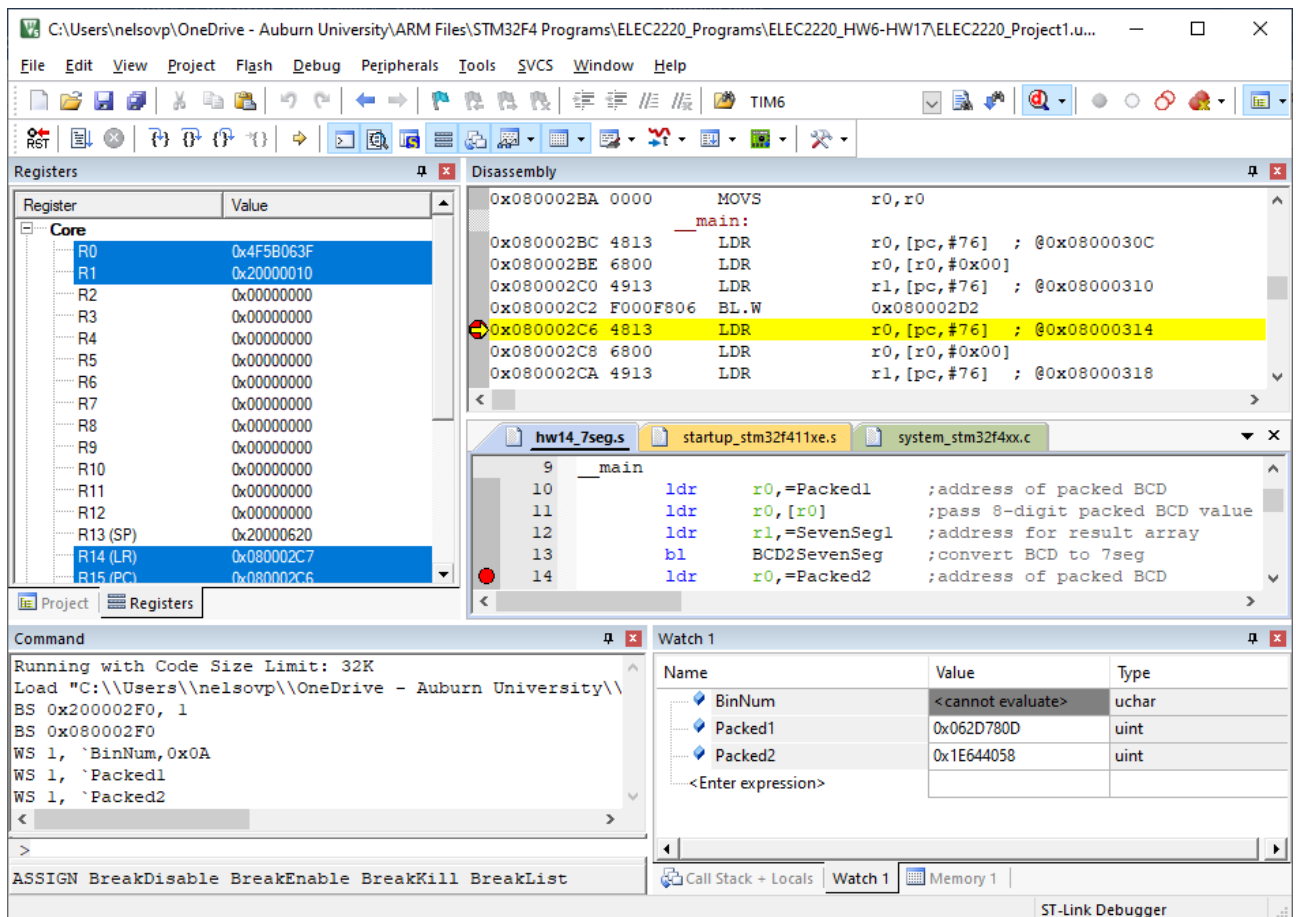
Click on "OK" for the pop up window showing "EVALUATION MODE, Running with Code Size Limit: 32K".

Open your uVision5 to full screen to have a better and complete view. The left hand side window shows you the registers and the right side window shows the program code. There are some other windows open. You may adjust the size of them to see better. Run the program step by step, you can observe the change of the values in the registers.



Click on the "Start/Stop Debug Session" from the "Debug" menu or click on the debug button to stop executing the program.

MDK uVision Debug Window



Debug Control:

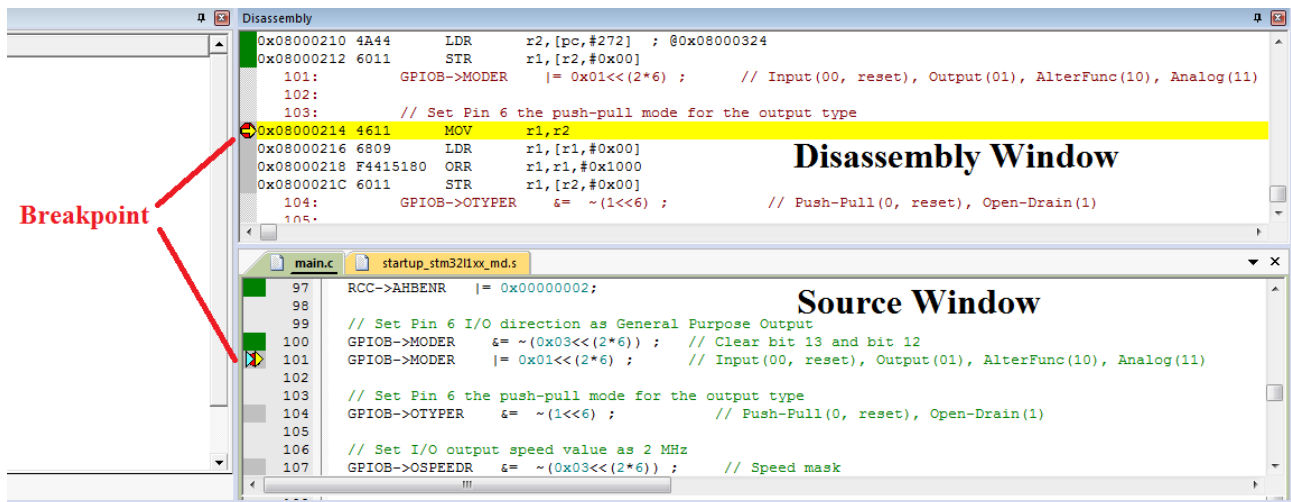
- Click the debug button to start the debug and click it again to exit the debug. You can use the breakpoint button to set a break point in either disassembly or source windows.
- STM32 allows up to six breakpoints during hardware debugging. When a program stops at a breakpoint, the corresponding instruction has not been executed yet.
- If the disassembly window is in focus, the debugger executes assembly instructions step by step. If the source window is focused, the debugger then steps through the source lines instead



- The following table summarizes commonly used debug control buttons.

						
Start Debug	Set a Breakpoint	Run	Stop Debug	Step In	Step Over	Step Out

- **Run:** Continues the execution from the current position until you click **Stop** or the program is paused by a breakpoint.
- **Step In:** Execute one step and enter the function if the current step calls a function.
- **Step Over:** Execute one step and run the function all at once if the current step calls a function.
- **Step Out:** Execute until the current function returns



We will analyze the program and see how it works.

It works with both the simulated target and the real circuit

board STM32VLDISCOVERY Board. I will demonstrate it in the lab for you.

8. ARM Instructions

Here are a few sample ARM Instructions for you to test out for this lab:

MOV R2, #0x76 ; Move the 8-bit Hex number 76 to the low portion of R2

MOV R2, #0x7654 ; Move the 16-bit Hex number 7654 to the low portion of R2

MOVT R2, #0x7654 ; Move the 16-bit Hex number 7654 to the high portion of R2

MOV32 R2, #0x76543210 ; Move the 32-bit Hex number 76543210 to the R2

LDR R2, = 0x76543210 ; Load R2 with the 32-bit Hex number 76543210

ADD R1, R2, R3 ; R1 = R2 + R3

ADDS R1, R2, R3 ; R1 = R2 + R3, and FLAGS are updated

SUB R1, R2, R3 ; R1 = R2 - R3

SUBS R1, R2, R3 ; R1 = R2 - R3, and FLAGS are updated

B LABEL ; Branch to LABEL

9. Lab Assignment

Write your first ARM assembly language program **MyFirstARM.s**. The program will execute the following instructions. You will run the program step by step, observe and answer the question after each statement.

```
MOV R2, #0x01 ; R2 = ?
MOV R3, #0x02 ; R3 = ?
```

```
; Other examples to move immediate values
MOV R5, #0x3210 ; R5 = ?
```

```
MOVT R5, #0x7654 ; R5 = ?
```

```
MOV32 R6, #0x87654321 ; R6 = ?
```

```
LDR R7, = 0x87654321 ; R7 = ?
```

```
ADD R1, R2, R3 ; R1 = ?
```

```
MOV32 R3, #0xFFFFFFFF ; R3 = ?
```

```
ADDS R1, R2, R3 ; R1 = ?
```

; specify Condition Code updates in CPSR

```
SUBS R1, R2, R3 ; R1 = ?
```

; specify Condition Code updates in CPSR

```
MOV R4, #0xFFFFFFFF ; R4 = ?
```

```
ADD R1, R2, R4 ; R1 = ?
```

; How did that operation affect the flags (Condition Codes) in CPSR?

```
ADDS R1, R2, R4 ; R1 = ?
```

; Please specify Condition Code updates

; and now what happened to the flags (Condition Codes) in the CPSR?

```
MOV R2, #0x00000002 ; R2 = ?
```

```
ADDS R1, R2, R4 ; R1 = ?
```

; again, what happened to the flags?

```
MOV R2, #0x00000001 ; R2 = ?
```

```
MOV R3, #0x00000002 ; R3 = ?
```

```
ADDS R1, R2, R3 ; R1 = ?
```

```

; Add some small numbers again
; and check the flags again.....

; Add numbers that will create an overflow
MOV    R2,    #0x7FFFFFFF      ; R2 = ?
MOV    R3,    #0x7FFFFFFF      ; R3 = ?

ADDS   R1,R2,R3                ; R1 = ?
; Check the flags in the CPSR?

```

10.